

| Wk08-Q06: PC ____ รหัส ____ ชื่อ-สกุล ____ |                             |                             |                             |
|--------------------------------------------|-----------------------------|-----------------------------|-----------------------------|
|                                            | <input type="checkbox"/> SS | <input type="checkbox"/> UU | <input type="checkbox"/> TT |
| Hardware                                   | MCS51                       | ESP32                       | ESP32                       |
| Tools                                      | Assembly or C or Basic      | C                           | MicroPython                 |
| Note                                       | Open Wwww without AI        | Open Wwww without AI        | Open Wwww without AI        |

Step1: ต่อ 4Digit 7Segment เข้ากับ CPU ทดสอบการทำงานด้วยการโปรแกรมให้แสดงรหัส นักศึกษาของตัวเอง

<https://shorturl.at/riQql>

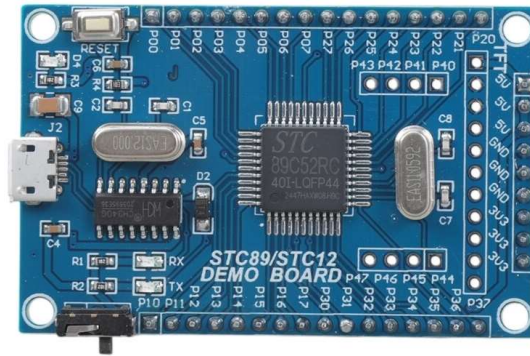
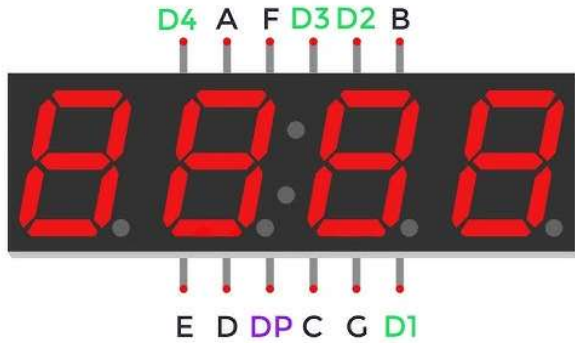
Step2: ปรับให้โปรแกรมทำงานแบบ Interrupts เพื่อทำงานดังนี้

- เริ่มต้นให้ค่าการนับเป็นรหัสนักศึกษา 4 ตัวท้าย
- รับ Interrupt ตัวที่หนึ่งเพื่อเพิ่มค่าการนับทีละ 2
- รับ Interrupt ตัวที่สองเพื่อลดค่าการนับทีละ 1

Read

- <https://wokwi.com/projects/389508770388175873>
- <https://randomnerdtutorials.com/micropython-interrupts-esp32-esp8266/>

# Test 4Digit 7Segment – STC89C52



```

1 pinSeg EQU P1 ; tgfedcba = p17 p16 p15 ...
2 pinPos EQU P3 ; p37 p36 p35 p34
3
4 ORG 0000H
5 JMP 0100H
6
7 ORG 0100H
8 CLR EA
9 MOV SP,#3FH
10
11 LOOP: MOV DPTR,#1234H
12 CALL Display4Digit
13 JMP LOOP
14
15 Display4Digit:
16 MOV B,#3 ; Start position.3(Left)
17 MOV A,DPH
18 CALL Display2Digit
19 MOV A,DPL
20
21 Display2Digit:
22 PUSH Acc
23 SWAP A
24 CALL Display1Digit
25 POP Acc
26
27 Display1Digit:
28 PUSH Acc
29 PUSH DPH
30 PUSH DPL
31 MOV DPTR,#SegmentTable
32 ANL A,#0FH
33 MOVC A,@A+DPTR
34 CPL A ; Active Low
35 MOV pinSeg,A
36 ORL pinPos,A
37 CLR A
38 DJNZ Acc,$
39 ANL pinPos,#0FH
40 POP DPL
41 POP DPH
42 POP Acc
43 DEC B
44 RET
45
46
47
48 SegmentPosition:
49 DB 80H,40H,20H,10H
50
51 SegmentTable:
52 DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H
53 DB 7FH,6FH,77H,7CH,39H,5EH,79H,71H
54
55 END

```

```

pinSeg EQU P1 ; tgfedcba = p17 p16 p15
pinPos EQU P3 ; p37 p36 p35 p34

ORG 0000H
JMP 0100H

ORG 0100H
CLR EA
MOV SP,#3FH

LOOP: MOV DPTR,#1234H
CALL Display4Digit
JMP LOOP

Display4Digit:
MOV B,#3 ; Start position.3(Left)
MOV A,DPH
CALL Display2Digit
MOV A,DPL

Display2Digit:
PUSH Acc
SWAP A
CALL Display1Digit
POP Acc

Display1Digit:
PUSH Acc
PUSH DPH
PUSH DPL
MOV DPTR,#SegmentTable
ANL A,#0FH
MOVC A,@A+DPTR
CPL A ; Active Low
MOV pinSeg,A
MOV DPTR,#SegmentPosition
MOV A,B
MOVC A,@A+DPTR
ORL pinPos,A
CLR A
DJNZ Acc,$
ANL pinPos,#0FH
POP DPL
POP DPH
POP Acc
DEC B
RET

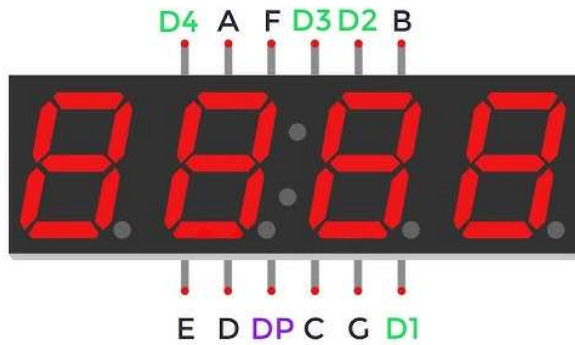
SegmentPosition:
DB 80H,40H,20H,10H

SegmentTable:
DB 3FH,06H,5BH,4FH,66H,6DH,7DH,07H
DB 7FH,6FH,77H,7CH,39H,5EH,79H,71H

END

```

## Test 4Digit 7Segment – C ESP32



```

1 // abcd efgt
2 const int segmentPins[] = { 17, 5, 18, 19, 21, 22, 23, 15 };
3 const int positionPins[] = { 13, 12, 14, 27 };
4
5 const byte digitPatterns[] = {
6     0b00111111, // 0
7     0b00000110, // 1
8     0b01011011, // 2
9     0b01001111, // 3
10    0b01100110, // 4
11    0b01101101, // 5
12    0b01111101, // 6
13    0b00000111, // 7
14    0b01111111, // 8
15    0b01101111 // 9
16 };
17
18 void setup() {
19     for (int i = 0; i < 8; i++)
20         pinMode(segmentPins[i], OUTPUT);
21     for (int i = 0; i < 4; i++) {
22         pinMode(positionPins[i], OUTPUT);
23         digitalWrite(positionPins[i], LOW);
24     }
25 }
26
27 void displayDigit(int digit, int position) {
28     byte pattern = digitPatterns[digit];
29     for (int i = 0; i < 8; i++) {
30         bool bitValue = !bitRead(pattern, i); // Active Low
31         digitalWrite(segmentPins[i], bitValue);
32     }
33     digitalWrite(positionPins[position], HIGH);
34     delay(5);
35     digitalWrite(positionPins[position], LOW);
36 }
37
38 void display4Digit(int temp) {
39     temp /= 10000; displayDigit(temp / 1000, 3);
40     temp /= 1000; displayDigit(temp / 100, 2);
41     temp /= 100; displayDigit(temp / 10, 1);
42     temp /= 10; displayDigit(temp / 1, 0);
43 }
44
45 void loop() {
46     int Counter = 1234;
47     display4Digit(Counter);
48 }
49

```

```

// abcd efgt
const int segmentPins[] = { 17, 5, 18, 19, 21, 22, 23, 15 };
const int positionPins[] = { 13, 12, 14, 27 };

const byte digitPatterns[] = {
    0b00111111, // 0
    0b00000110, // 1
    0b01011011, // 2
    0b01001111, // 3
    0b01100110, // 4
    0b01101101, // 5
    0b01111101, // 6
    0b00000111, // 7
    0b01111111, // 8
    0b01101111 // 9
};

void setup() {
    for (int i = 0; i < 8; i++)
        pinMode(segmentPins[i], OUTPUT);
    for (int i = 0; i < 4; i++) {
        pinMode(positionPins[i], OUTPUT);
        digitalWrite(positionPins[i], LOW);
    }
}

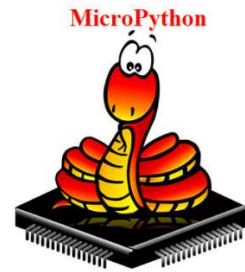
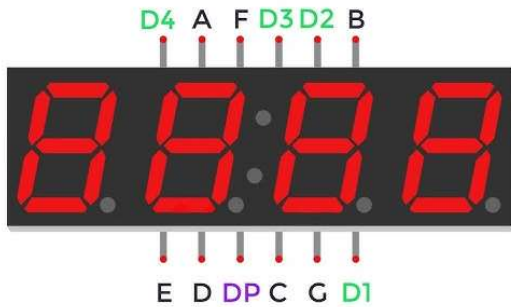
void displayDigit(int digit, int position) {
    byte pattern = digitPatterns[digit];
    for (int i = 0; i < 8; i++) {
        bool bitValue = !bitRead(pattern, i); // Active Low
        digitalWrite(segmentPins[i], bitValue);
    }
    digitalWrite(positionPins[position], HIGH);
    delay(5);
    digitalWrite(positionPins[position], LOW);
}

void display4Digit(int temp) {
    temp /= 10000; displayDigit(temp / 1000, 3);
    temp /= 1000; displayDigit(temp / 100, 2);
    temp /= 100; displayDigit(temp / 10, 1);
    temp /= 10; displayDigit(temp / 1, 0);
}

void loop() {
    int Counter = 1234;
    display4Digit(Counter);
}

```

## Test 4Digit 7Segment – MicroPython ESP32



```

1 from machine import Pin
2 import time
3
4 # Pin Mapping: a, b, c, d, e, f, g, dp
5 seg_pins_numbers = [17, 5, 18, 19, 21, 22, 23, 15]
6 # Position Pins: Dig1, Dig2, Dig3, Dig4
7 pos_pins_numbers = [13, 12, 14, 27]
8
9 segments = [Pin(p, Pin.OUT) for p in seg_pins_numbers]
10 positions = [Pin(p, Pin.OUT) for p in pos_pins_numbers]
11
12 # รูปแบบตัวเลข 0-9 (Binary เหมือนใน C++)
13 digit_patterns = [
14     0b00111111, # 0
15     0b00000110, # 1
16     0b01011011, # 2
17     0b01001111, # 3
18     0b01100110, # 4
19     0b01101101, # 5
20     0b01111101, # 6
21     0b00000111, # 7
22     0b01111111, # 8
23     0b01101111 # 9
24 ]
25
26 def display_digit(digit, pos_index):
27     pattern = digit_patterns[digit]
28
29     for i in range(8):
30         bit_value = (pattern >> i) & 0x01
31         segments[i].value(not bit_value)
32
33     positions[pos_index].value(1)
34     time.sleep_ms(5) # หน่วงเวลาเพื่อให้ตามองเห็น
35     positions[pos_index].value(0)
36
37 def display_4_digit(number):
38     number %= 10000
39     display_digit(number // 1000, 3)
40     display_digit((number // 100) % 10, 2)
41     display_digit((number // 10) % 10, 1)
42     display_digit(number % 10, 0)
43
44 # Main Loop
45 counter = 5678
46 while True:
47     display_4_digit(counter)
48 
```

```

from machine import Pin
import time

# Pin Mapping: a, b, c, d, e, f, g, dp
seg_pins_numbers = [17, 5, 18, 19, 21, 22, 23, 15]
# Position Pins: Dig1, Dig2, Dig3, Dig4
pos_pins_numbers = [13, 12, 14, 27]

segments = [Pin(p, Pin.OUT) for p in seg_pins_numbers]
positions = [Pin(p, Pin.OUT) for p in pos_pins_numbers]

# รูปแบบตัวเลข 0-9 (Binary เหมือนใน C++)
digit_patterns = [
    0b00111111, # 0
    0b00000110, # 1
    0b01011011, # 2
    0b01001111, # 3
    0b01100110, # 4
    0b01101101, # 5
    0b01111101, # 6
    0b00000111, # 7
    0b01111111, # 8
    0b01101111 # 9
]

def display_digit(digit, pos_index):
    pattern = digit_patterns[digit]

    for i in range(8):
        bit_value = (pattern >> i) & 0x01
        segments[i].value(not bit_value)

    positions[pos_index].value(1)
    time.sleep_ms(5) # หน่วงเวลาเพื่อให้ตามองเห็น
    positions[pos_index].value(0)

def display_4_digit(number):
    number %= 10000
    display_digit(number // 1000, 3)
    display_digit((number // 100) % 10, 2)
    display_digit((number // 10) % 10, 1)
    display_digit(number % 10, 0)

# Main Loop
counter = 5678
while True:
    display_4_digit(counter)

```